

# Lec 22 Working of Probabilistic Decoder, ELBO, Loss Function of VAE

## Lec 22 Working of Probabilistic Decoder, ELBO, Loss Function of VAE

---

|                 |                                                        |
|-----------------|--------------------------------------------------------|
| <b>Lecture</b>  | 22 / 63 + intro                                        |
| <b>Duration</b> | 44:39                                                  |
| <b>Captions</b> | English (auto-generated)                               |
| <b>Words</b>    | 6,759                                                  |
| <b>Video</b>    | <a href="#">09A4OkjUjt0</a>                            |
| <b>Playlist</b> | <a href="#">Fundamentals of Generative AI and LLMs</a> |

---

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

### Transcript

**[00:00]** Hello everyone, welcome to the lecture series on fundamentals of generative AI and large language models. We are in the week three. In the previous session, we had seen the encoder part of VAE variational autoencoder. And in this particular session, we will look into the decoder part of VAE. We will look into ELBO which is called an evidence lower bound. And towards the end, we will connect it to the loss function of VAE. This is an important session because in this particular session we will be covering some of the concepts which we have raised even in encoder part and uh there were some questions which were not very clear in encoder.

**[00:45]** Those things will be answered here and towards the end of this particular session you will get a complete idea of how the encoder works how the decoder works. So if you are listening to this particular session, I request you to see the previous session also once and uh only then you will have a complete clarity of the overall working mechanism of the variational autoencoder. So let's get started. This is what we have seen. The encoder is going to give us the output. What output? The

encoder is going to give the uh  $\mu$  and then the mean and then the variance of an approximate posterior distribution. This is what the encoder is going to give us the output. Right? From our previous session towards the end when we have

**[01:33]** seen a numerical example, we had also seen how  $z$  is sampled from this distribution. The theory says  $z$  is sampled from a distribution. We have also seen one with the mathematical example that we had seen in our encoder part of VAE. So once the encoder gives mean and variance of an approximate posterior distribution, the next uh uh thing is  $z$  is sampled from this distribution. So this also we have seen in the previous one.  $z$  is sampled from this distribution. And the meaning of sample is randomly picking one number from this distribution. Now we have  $z$ . This is what is our previous part. Now once we obtain  $z$  is it not  $z$  is passed as an input to a decoder. So we have the decoder. Now

**[02:29]** where  $z$  is passed as an input to a decoder. What is the role of a decoder? So what is this  $v$  is all about? It is used to generate a new data or for a reconstruction primarily for an generating a new sample or new data. So here um decoder is to um either give to generate or reconstruct the data from where from this  $z$  as an input to a decoder. Therefore decoder is a neural network that takes  $z$  as an input and then it gives us the output. What is that output? What is the role of a decoder? You remember this is a probabilistic uh model which means we have a probabilistic encoder and then a probabilistic decoder. If that is the case, decoder also has to output um you know the parameters and those parameters

**[03:23]** are likelihood distribution. What is likelihood distribution? What is the probability that we get  $X$  given  $z$  as an input? Right? So decoder is a neural network that takes  $z$  as an input and then outputs the likelihood parameters um uh you know parameters of likelihood distribution. Now these uh parameters depends on what kind of data that you are going to reconstruct whether you are going to reconstruct the continuous data or whether you're going to reconstruct the binary data depending on that the types of parameters are going to vary.

**[04:05]** So the types of parameters produced by the decoder mainly depends on the type of data being generated. So the first possibility is it can be a continuous data. If it is a continuous data then the uh output of a decoder is mean and variance. So uh then uh uh  $\hat{x}$  or the generated sample uh is sampled from this distribution. So uh let me make it very clear. So decoder has to output give give us the output that output depends on the type of data that the decoder is reconstructing or type of data the decoder is generating. If the uh decoder is reconstructing or generating a continuous or real values then the decoder is going to give us the output mean and variance. Okay. And theoretically we say that this  $\hat{x}$  is

**[05:03]** being sampled from this distribution is sampled from this uh distribution which is having mean and variance. Uh so here decoder is being defined as what

defined as gaussian right. So what you have to remember here is suppose if it is an real values or a continued values then it is modeled as gaussian and decoder is going to give us the output mean and variance and theoretically we say that  $x$  is sampled from this distribution but practically whatever is the output that is mean and variance that is the reconstructed one. So that we will see probably in the next session where we are solving one entire problem. But now theoretically you uh understand this as whenever we are going to get an continued um require a continued values

**[05:57]** then a decoder is going to output mean and variance. At the other side what is the uh what if we want and binary values. If it is a binary values what is that the decoder has to output? the decoder has to um output the probability for each of the feature or a pixel. What is the probability that each feature is either zero or one? So which means uh it is a bernoulli distribution. So uh it is defined as bernoulli distribution and the um  $x$  that is um sampled from this particular distribution.

**[06:36]** So uh this is what is the decoder part. So let us take one example like how we had understood the encoder with one numerical example towards the end. Let us also understand decoder with one numerical example that numerical example is this. Okay we um let us understand this very carefully so that you know the complete overall working mechanism of decoder will be answered in this particular slide only. Now let us take this input. What is this uh uh input here? The input is original  $X$  and here we have totally having five features.

**[07:14]** Therefore we have five uh uh here. Okay. Now once this is done you will be passing it to an um encoder. So once you pass it to an encoder what is that we have understood encoder is going to give us the output. What output it is going to give? It will give mean and variance. So let us assume since the latent space size is two uh here uh space size is two [snorts] we are going to get two mean values and two variance values. Let us assume this is two mean and this is two variance values. So let me erase it.

**[07:53]** Since the uh size here is two, we are getting uh two mean and two variance. Even though if it is the log variance, convert it into a variance. So it is two variance. Then what is that you are going to do? Since  $z$  is two, we have  $z_1$   $z_2$ . So  $z_1$  is sampled from where? From this and this distribution. And  $z_2$  is sampled from this and this uh uh distribution. So far we are clear and we have seen in the previous session what do you mean by sampling? Sampling is randomly picking one uh value. So now um whatever the value that randomly you pick that will be this value and  $z_2$  when it is sampled from this distribution that is this value. Now I'll ask one question to the learners here. Uh whenever you sample one value

**[08:49]** is that value fixed? What is your answer? The answer is no. The value is not fixed. Right? If the value is not fixed that means you will get uh one more um possibility. This can be that one more possibility and this is one more possibility for  $Z_2$

like this. How many such values are possible? Infinite. There is no end right? Infinite such possible values. Are you understanding? when I say infinite such possible when I sample one value I can get the value as 0.45 I'll get 0.46 46 or I can get 0.55 0.54 0.51 there is no end um when I sample a value so from this when I pick randomly one value probably first time I would have got this value second time I'll get this value third time I'll be getting this so there can be any number

**[09:42]** of such infinite such a possible values so those are all these uh uh values is this point clear um let me pause and then tell you Once again because this is one important uh um lecture where the entire VAE theoretically you'll be able to you'll be understanding. We have taken an input  $X$  is passed to an encoder. The output of an encoder is mean and variance. Right? Since the latent space is two, we are having two mean two variance. Now  $Z_1$  is sampled from this distribution and  $z_2$  is sampled from this distribution. Sampling here is randomly picking one number.

**[10:26]** When I say randomly picking one number, for a given input  $X$ , there can be infinite such  $z_1$ ,  $z_2$  and so on up to  $Z_N$  where  $n$  is an infinite value. You will have infinite such values. What do you mean by infinite such values? Probably initially it will be this and this will be 0.8. Then it can be 0.55. It can be uh 0.22. Then it can be 0.46 and this can be minus 0.2. It can be any like this. There can be any number of such values which are possible. For example, let us assume there are three samples where we have obtained 0.56 minus 0.11 as the first sample which is  $Z_1$  [snorts] and the second sample is  $Z_2$  which is 0.43 and -0.32.

**[11:26]** Third sample  $Z_3$  is 0.61 and minus 0.24. What do you mean by these? when you randomly pick the values from  $z_1$  and  $z_2$ . This is the first sample, second sample and third sample. Are you convinced up till this point? Is this uh clear so far? I hope um yes, answer is yes. Okay. Now once you complete this, what is from our just the previous slide? What did we understand? Once we get this  $z$  that is latent representation, we are passing it to a decoder. So which means  $z$  is passed now to a decoder right when  $z$  is passed to a decoder what is that decoder has to output that depends on what kind of an input here input is this if uh input is this uh that is continuous value what output it is going to give us

**[12:18]** is the mean and uh variance value because it is defined as a gaussian okay now uh how is that decoder works when  $z$  is passed to a decoder. Decoder gives us  $\hat{X}$ . Okay. So is it not this is the working mechanism of a decoder. Pass  $Z$  and then we get  $\hat{X}$ . Okay. Now how many such  $z$ s are there here? There are three such possible. Okay. Just for an example I'm telling you. So now here for an example we have taken three such samples.  $z_1$ ,  $z_2$  and  $z_3$ . Now  $z_1$  when you pass it to an decoder is it not? it is going to give us this uh this value agree right now u um I'm just giving one example where  $z$  is passed to an decoder and then decoder has to give us this  $\hat{x}$  and  $\hat{x}$  how many inputs are

**[13:15]** there there are five so definitely decoder also has to give five uh values we have seen this from our encoder your architecture will be such that towards the end you will be having five um uh feature five neurons and definitely you'll be getting five such values. So let us assume those five values are these. When you pass Z1 you got this. Now there are three such different samples. So when uh and you see this vector is different from this vector agree yes. Now when you pass Z2 to a decoder is it not you are going to get slightly different values. Let us assume those values are these. Now when Z3 is passed as an input to an decoder, decoder has to out give us the output.

**[14:04]** Let us assume the decoder has given us this output. Look at this very carefully. Let me clear the screen and then uh look into this. What is that we have done? uh we have taken the input and then we have obtained the latent space and latent space has infinite such a possible values for the given input. Agree what do you mean by infinite such whatever we are seeing here when you sample one value from that particular distribution you will get infinite such values which means there are infinite such samples. Now uh uh every of zed when it is passed to an decoder decoder has to give us the output. What output decoder is giving? Uh here because the input is five uh features decoder also has to give five. Our architecture is

**[14:56]** such that the last layer is having five neurons. So five output we are getting. So when you pass a zed one is it not we got this output. when we pass a zed 2 is it not uh this I'm just giving as an example just for an nu uh you know visualization for an understanding when we pass Z3 to a decoder we are getting this now when you look into all of these what is one common we have is it not all of these features um the values that we have obtained um uh  $x_1$  hat  $x_2$  hat and  $x_3$  hat all these are very much closer to the original X can you see there is a slight variation but mean and around it is playing around those features only why there is a variation there is a variation because there is a variation

**[15:47]** in uh uh zed that we are passing to a decoder when there is a small variation in the decoder input definitely there is a small variation even in the output also so is it not what is generative model generative model is it has to generate a new data Yeah, original data is this and over a period of time by the end of a decoder after once the training everything is done is it not we are getting an output here definitely it is different from the original uh uh input. So so far we were focusing on reconstructed but what we are now focusing is and generating a new sample. How is that we are generating a new sample? we are generating a new sample by learning the uh underlying distribution once that underlying

**[16:38]** distribution is learned properly. Now you can ask where is that learning happened. So uh initially when the training is going on probably the mean and variance will be very different values over a period of time during back propagation you would have updated those weights and then over a period of time whatever the values that

we have obtained here will be the optimized values with those optimized values when you pass it to an decoder definitely whatever the reconstructed or generated values will be very much closer to the original one. So um did you get this concept? So here what exactly has happened? input for a given input. Uh there are multiple such latent variables um are possible. uh and there'll be

**[17:28]** since it is a sampled uh from that distribution each time you sample the values will be different when the values are different definitely that different values see this is the first one second and third each time when you are getting a different samples when you pass it to an decoder decoder is going to give us the different uh um output but it is not uh although it gives different but it will be very much closer to the original input that's what the reconstructed outputs are slightly different because different latent vectors were sampled.

**[18:03]** However, all of them will remain uh similar to the original input. Now this is what is the overall working mechanism of an decoder. Now let us slowly uh one by one we will answer to all our previous uh questions. Now see uh do you agree to this? There are infinite many such possible zeds. Agree? Yes. Uh you can keep sampling any number of times and each time you sample the values will be different. So infinite many such zeds are possible. Now u and for one observed for a given input x whatever the input that is given here this corresponds to many possible latent variable representation zed. Do you agree? The input what is given to us is this for this input you will be getting many such zeds and there are infinite such zeds

**[19:00]** that are uh possible. So this is what is being explained so far. Now there is one question that we will be um asking to ourself. What is that question? Now this is the input that is given right now for this there are many such zeds that are possible. So many zeds are there which are sampled from that distribution. Now each uh zed when you pass it to decoder you are getting some output. Now there is one uh uh question. What is that question? uh for for a given input whatever is the training sample X which latent representation zed among so many zeds which zed can produce or has produced this sample through the decoder which zed has given us the output can you tell one particular zed this is the

**[19:58]** zed which has given us the output no we cannot tell why we cannot tell because it is just not one zed Zed you you have multiple such zeds which are actually corresponding to um you know we we are getting that output. Now if these are the different outputs that we have obtained now I'll ask the question for this input whatever we have given which zed has produced the output what is your answer is no it is not one such a zed you have infinite such a zeds right so in order to answer to this particular question uh now slowly we are connecting certain dots in order to know which latent variable zed would have generated this observed input X or the given input X.

**[20:47]** If you want to know the answer for this that can be calculated only by using the true posterior according to the base rule. This requires in order to answer to this particular question it requires computing this  $P$  of  $Z$  given  $X$ . Right? Now we know that  $P$  of  $Z$  given  $X$  can be computed using base rule with this. Sometime back in our previous session we have seen the denominator  $P$  of  $X$  is uh written in this way. Now what is this?

**[21:19]** Now this slide is very important. Uh what exactly um is this slide is all about is now some time back you had a question that okay you got the output. Now which is  $Z$  has given you this output. We cannot tell one particular  $z$  has given because there are multiple such  $z$ s are um possible. In order to get that answer you need to compute  $p$  of  $z$  given  $x$ . To compute this there is a problem that denominator term is difficult. Why denominator term is difficult. The difficulty is actually here. This requires the integral I mean the this is the integral um where all possible latent variable  $z$  that would have generated the observed. You have to consider all of those because it is just not one  $z$  that has given you the

**[22:10]** output. There are multiple such  $z$ s that has given you the output. So you have to consider all of those. You have to take the summation of all of those. So for a neural network um especially at a decoder side this integral is actually difficult. It is intractable. A technical term that we'll be using here is intractable which is difficult because of this difficulty. We never use an  $p$  of  $z$  given  $x$ . We always go for an approximation.

**[22:43]** Okay. So uh I hope you are able to connect the dots. Now um if if at all if you want to answer which  $z$  has uh is responsible for whatever the output that we have got we cannot answer that because there are many such  $z$ s that are that would be possible for getting that reconstructed one. In order to answer you have to consider all of those. How many you will consider? Is there any limit for it? No. Therefore for a neural network decoder this is difficult. Further let us understand why uh it is  $p$  of uh  $z$  is an  $u$  priority distribution which we know that we have assumed it to be a  $g$ shian  $g$ shian is having these parameters. Okay. Now uh slowly you see this uh every  $z$  that is the latent um in

**[23:34]** the previous slide if you go back okay if you look into this there are many such  $z$ s that are uh possible. Now uh let's uh go back and then look into this particular question. For every possible  $z$ , what is the decoder gives? Uh what is the probability uh look into this? What is the probability that this  $z$  could generate this  $x$  correct? Now you have obtained some uh uh output you from our previous slide when  $z$  is passed to an decoder you have got some  $\hat{x}$ . So u once you get this the model is going to assign some probability right it can be 2.1 so it is assigning some probability so for each of the  $z$  the decoder is going to assign the probability what is the probability what is the probability that you would have

**[24:37]** got this  $x$  from this  $z$  or uh this  $z$  could generate the observed  $X$  read in whatever it is one and the same it is going to assign the probability okay next uh uh there is one more term what is that one more term you see this what is this uh uh term I'm I'm explaining why  $P$  of  $X$  is difficult so for every of  $Z$  decoder is going to assign one probability what is that probability okay if you have obtained what is the probability that you obtained this  $\hat{X}$  or  $X$  from this latent uh representation  $z$ . Okay. Or this  $z$  um would generate the observed  $X$ . What is the probability that this  $z$  would have generated that  $X$ ? That is what is the first term. Now what is this second term? Now for each latent vector also there is some prior

**[25:39]** probability. What is the meaning of this? for each latent  $u$  vector. Now can we say this is one latent vector? This is one latent vector. E for each of these latent vectors this is one sample latent vector. This is one. For each of these latent vector there is one probability that uh the model will assign. What is that probability? If the values which are there in this vector, if those values are very much closer to zero, which means they are very much closer to origin, then it is going to assign high probability, high prior uh density. Uh here  $Z_1$  um is going to get since it is very much closer to origin, it is going to get relatively high prior density.

**[26:31]** Okay. Now since here the values are not very much closer to zero then it is going to give a very low priority density right now which means here it is going to assign some value here it is going to some uh it is going to assign some value. What is the total contribution? Is it not? The total contribution is the product of these two product of this and the product of this. Right? The total contribution of each possible latent vector is this. And when you have to compute  $P$  of  $X$ , it is the integral. It is the sum of all such values. How many such values we have?

**[27:17]** Infinite. So denominator is  $u$  um you know difficult term here. That is the reason we uh encoder never aims to learn  $P$  of  $Z$  given  $X$ . Instead encoder learns  $Q$  of  $Z$  given  $X$ . Now you can ask by using certain uh learnable parameters. Fine. Now you can ask uh ma'am what is the difference between this and this? Here also it is the probability. Here also it is the probability. In what way these two are different? Look this is actually an posterior um you know approximate posterior which when you pass it to an neural network it is just giving you the output um  $\mu$  and then the variance.

**[28:09]** Whereas if your aim is to learn this what is this? It is a base rule. What is base rule? Base rule says you should have this, you should have this and you should have this. Now if you have this as your base rule, the denominator is very difficult because it requires the integral of infinite many such all possible latent variable  $z$ . So there is no end for that denominator. That is the reason. Now you can ask is it not the problem in  $q$  of  $z$  given  $x$ ? No. the  $Q$  of  $z$  given  $X$  is not following this basian rule.  $Q$  of  $z$  given  $X$  follows the gshian where it will tell that I should only output mean and variance but whereas  $P$  of  $Z$  given  $X$  follows the basian. If it follows the

basian it has it has the denominator term in the

**[28:59]** denominator term you have that integral over all the possible latent variables. What do you mean by all possible latent variable? Now if  $Z$  if decoder has given reconstructed  $z$  which is uh reconstructed output  $X$  then which  $z$  is responsible for getting that output. It is not one such  $z$  it is many such  $z$ s. If that is the case you have to consider all of those. That is the reason encoder always follows um approximate prior um distribution approximate posterior distribution but not the true posterior. This is uh the answer for what was the question raised in our encoder part. This is the slide where you will get an answer most important uh uh slide where um this is difficult to compute. Why? because it

**[29:56]** has um uh this term. What is this term? Uh for every possible  $z$  decoder gives what is the probability that this  $z$  would have generated this  $x$  and the second for every of the  $z$  there is one um you know uh prior density that we are assigning. How are we assigning? If that  $z$  has all the values closer to zero which is closer to origin um then we will get the higher um priority density. If the values are far from the origin that is the values which are not zero then we get the low prior density. Then for one  $z$  what is uh the total contribution? The total contribution is this like that many such  $z$ s are there.

**[30:43]** So it is the integral which is the addition of all of those. That is the reason instead of enforcing the encoder to learn this, we are enforcing the encoder to learn this part. Now um don't get confused. The difference is only  $P$  and here it is a  $Q$ . No, here it is a true posterior.  $P$  is a true posterior where it is following the base. Whereas here it is forced to follow the gshian. Because of that it is uh outputting it is giving us the output mean and variance. I hope this is uh clear to you. One of the most important um concept. Okay let's proceed. Now we are slowly moving towards the loss function.

**[31:30]** Now uh what is the loss function in um VAE? You remember um we have uh um priority distribution as gshian then encoder  $q$  of  $z$  given  $x$  is gshian who is ensuring then we used the term kullback divergence. So slowly let us move towards that. Now suppose the training data has eminest handwritten digit data set. Uh if if at all if the model is very well trained what is our ultimate outcome? uh what do you think are ultimate? What what what is that you are expecting from the model? If the model is very well trained, if um you know all those u um distributions are learned properly um reconstruction laws, everything everything has been learned properly, the training went very well.

**[32:18]** If that is the case, what is our ultimate expectation? If it is really in handwritten uh uh digit the model should give higher uh uh probability uh to that particular uh uh sample. What do you mean by higher probability is it should say confidently that the model um you know it is an handwritten one. The model should give high

probability to um you know digitalike images. If there are some noisy, if it is not an handwritten one, if there is some disturbance for those images or the data, it should assign the lower probability. This is what we expect. Then only we say that the model has learned very well the underlying distribution. Do you agree to this? I'll pause here and then I want you to think about what the statement I

**[33:04]** have made. After once the training is completed, what is that we are expecting from the model? uh once the training is uh done it means the model has learned the underlying distribution. If it has learned the underlying distribution if it is an digit like uh images hand written a digit like images the model should give higher probability to it or otherwise it should assign the lower probability which means our aim is to maximize this. This is our aim. It has to maximize the log of  $p(\theta|x)$ . It has to but just some time back we have seen that um you know computing this is very difficult because it has to check all possible latent variables uh that would have generated this  $x$ . So it is very difficult right? So which means

**[33:52]** this is not our aim. What is our aim then? So instead of directly maximizing  $\log p(\theta|x)$  can we maximize some other objective? What is that other objective is elbow which is evidence low bound. So uh instead of directly maximizing  $p(\theta|x)$  we are maximizing a simpler objective and that objective is um elbow. Now what is elbow? Uh is computing the lower bound of  $p(\theta|x)$   $\log p(\theta|x)$ . Therefore instead of VA instead of maximizing  $\log P(\theta|X)$  it maximizes the elbow. VAE maximizes the elbow. So uh what is that we have uh we are understanding from this once the training is completed once the model has learned properly the aim here is for all um correct samples it has to give higher

**[34:52]** probability for all incorrect or for all you know noisy one it has to give the lower probability which means our aim is to maximize this. If our aim is to maximize this problem is it cannot be computed because of all possible latent variable  $z$  that would have generated  $X$ . So therefore we cannot keep this as our objective. Our objective should be maximize the elbow. Now let us understand what is elbow. Elbow formula is this where the first one is reconstruction. What is the reconstruction? Uh um how do you get this uh uh um one is it not from  $z$  you have sampled uh what is the role of an encoder? Encoder would have learned uh this distribution. Once encoder has learned this distribution,  $z$  is

**[35:42]** sampled from this distribution. Yes. Once  $z$  is sampled from this distribution, that sampled value is given to an decoder. And what is the role of a decoder? decoder has to give the maximum probability for that particular uh sample. Right? So if the decoder samples  $z$  from this um appropriate posterior can the decoder reconstruct the original input  $x$  with high probability which is nothing but an reconstruction right so once again this in a very simple uh term I'm explaining  $z$  is sampled uh encoder has learned mean and variance once mean and variance is learned properly then once mean and variance is learned properly then  $z$  is sampled from

that distribution. If  $z$  is sampled from that distribution when

**[36:37]** that  $z$  is given to an decoder decoder has to output as again mean and variance if it is a continued values. So if this learning is proper if encoder learning is proper definitely reconstruction is proper. So the first term here is a reconstruction term and if it is an continued values here in this particular place we'll be using mean squared error. Suppose if it is an binary in this particular reconstruction place we'll use binary cross entropy. Is this part clear? Um you can take some time to analyze what is being explained so that um you you are with me up till this point. I hope this is clear to you.

**[37:20]** Right? Okay, what is the second term? Second term is KL divergence. Uh from our previous session, we have seen what is um from our first session in week three, we have seen what is KL divergence. So what is K scale divergence here? one you remember from our encoder session uh prior  $p$  of  $z$  is encoder I mean is gshian I'm sorry  $p$  of  $z$  is gshian then encoder has to learn some distribution instead of learning some complex we have assumed again it to be gshian so there are two gshians so the scale divergence will ensure that both um uh you know encoder distribution remain closer to a prior distribution pset Okay. So this is what is elbow.

**[38:08]** Elbow is uh where you have an um reconstruction and you have your KL divergence term. In this the very first term that is reconstruction is called uh you know the log likelihood. In VAE loss the first term is called log likelihood. And what is the log likelihood? We always expect the model should give the higher probability for the um uh you know observed data which means our aim is maximum but neural network how it is being trained always it has to minimize the error. If it has to minimize the error, it cannot be maximum. It has to be minimum. How? From maximum you'll convert to minimum. Minus is it not?

**[38:55]** From max of some function if you want to convert to minimum it is minus right. So we convert max into minimum. That max is converted into minimum with minus. So once again I'm repeating the first term is called the log likelihood. And what is that? We always expect it has to assign higher probability for the observed data which means maximum. The first value should be max of but neural network is always trained to minimize what is uh that not maximization minimize the loss. If it is a minimization max function is converted into minimum with minus. Therefore you see how the loss function is the loss function now is uh the minimum of it.

**[39:44]** So minus of this whole term if you give minus first term minus of minus becomes plus. So this is what is the total loss function uh or the VAE loss function. Important slide once again. Um so um elbow says um you know reconstruction and then the regularization term but um first term is maximum we don't want maximum we want to minimize therefore max is converted into minimum in the process of converting the first term is getting negative term minus of this is the positive value therefore

the va loss function the first term is a reconstruction ruction. If it is an um continued value or real values, this first term becomes mean squared error. If it is an binary, the first term becomes binary cross entropy,

**[40:47]** right? And the second is scale divergence. What is scale divergence? Kale divergence is represented in this form because the first is this is gshian which is written in this way. This is a gshian which is written like this. Right? Now because these two are gshian in a closed form we are writing in this uh uh way. Why it is called closed form? So that you can substitute some values and get some number. So in the closed form this equation equivalent is whatever we have written here. So in our previous session we have already seen how this equation is derived. Okay. So this is the um you know uh closed form KL divergence when both um so um distributions are gshian kl divergence has a closed form expression written in

**[41:41]** this way. This is all about the VA where we have started with the um encoder and there was some questions which were still not answered. Then we moved to decoder. In the decoder we have overall understood how the working mechanism. Then we started answering one by one why um true posterior cannot be used. There is one uh drawback here which is still not a complete VA. Uh there is one drawback here. What is the drawback? Can you tell how zed is sampled? Zed is sampled from this distribution. Now uh this is what is the KL divergence which encourages encoder distribution to remain closer to prior distribution.

**[42:24]** Fine. Now how is zed uh sampled? Zed is sampled from uh the distribution. Right? Which distribution? Thus this what we have seen in both the examples. Zed is sampled from this distribution. And each time when you sample it, you're getting a different values. Right? Each time when you are getting a different value, is it not? There's lot of randomness. When there is a randomness during a back propagation how is that you will back propagate through this one uh uh part so that you can update the weights.

**[42:58]** Randomness always breaks the gradient flows right. So if that is the case how will you make the back propagation? Back propagation is impossible in this scenario. Right? So this is important concept once again since zed is sampled from this distribution and since this distribution each time you sample you'll be getting a randomly a different value there is a randomness operation randomness always breaks the gradient flow which means we don't have one fixed formula to get that zed zed is sampled from normal distribution or a goshian one so because zed is not having a proper formula there is no way that you can back propagate it. So which means architecture is still not complete architecture. So this led to a question

**[43:49]** called uh uh you know reparameterization trick which will be our point of discussion in our next session. In the next we are going to discuss the reparameterization trick and then we will see if you look into the last two sessions uh this session

and the previous session. Nowhere we have seen the architecture of VAE. Only I started explaining you what is encoder, what is latent space, what is decoder. But we have nowhere seen a solid architecture diagram because after we complete the reparameterization trick, we will see what exactly is the architecture of VAE.

**[44:28]** So uh with this we have come to this particular uh uh end of this particular uh session. Thank you.